

VIGIA M7 Event Logging Plan

VIGIA M7: Event Logging + Database + Hazard Map

Document version: 1.0
Date: 2026-06-17
Project: VIGIA ADAS Edge Node (vigia-raspi)
Status: Engineering plan — pending implementation
Prerequisite: Milestone M6 multimodal sensor fusion (complete)

Executive summary

M7 turns fused pothole detections into a **durable, queryable hazard platform** — not a flat log of every frame. Each promoted detection becomes a versioned **observation**, syncs to a **PostGIS server**, clusters into **hazard entities**, and feeds a **hazard map UI**.

Hardware baseline (current BOM): No NVMe SSD. Pi buffers events in RAM (`/dev/shm`); the **server database is the source of truth**. SD card is boot media only — not for event logging.

1. Current state

Area	Status
Multimodal fusion (RRI + ISS + GPS)	Done — <code>src/fusion.cpp</code> , <code>src/coordinator.cpp</code>
Structured hazard output in production	Missing — <code>publishResult()</code> is debug stdout only
Event persistence / database	Not started
Hazard map API / UI	Not started
Secure cloud ingest	Designed in docs; not implemented

Related design references in repo:

- `.claude/design/02_ros2_node_contracts.md` — `HazardEvent.msg` schema
 - `docs/hardware-architecture.md` — JSON uplink, mTLS, long-term NVMe target
 - `.claude/design/05_anti_death_and_depin_contracts.md` — server attestation pipeline
 - `docs/sensor-fusion-plan.md` — milestone roadmap (M0–M7)
-

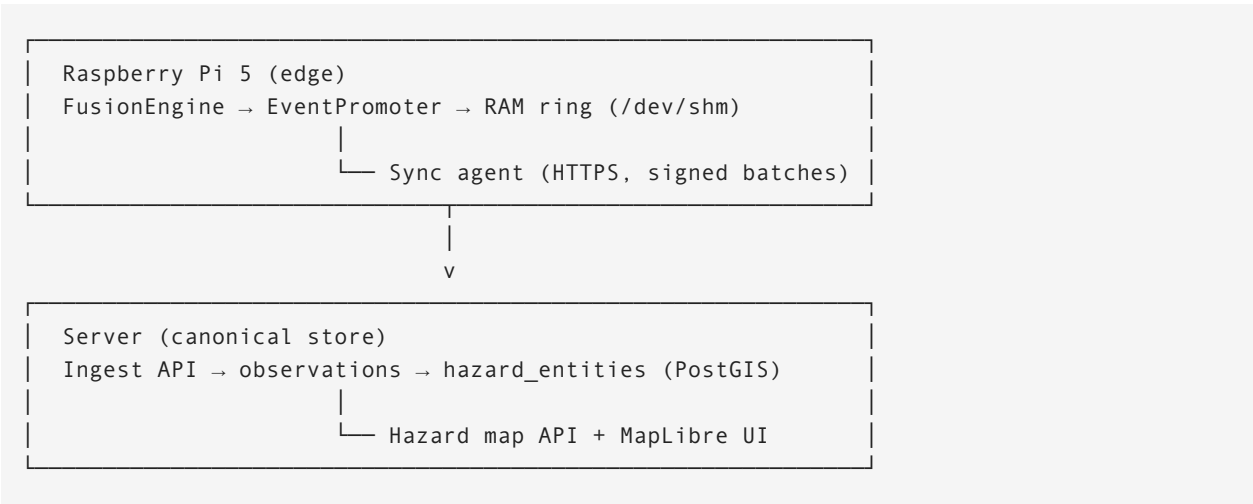
2. Why not a simple event log?

Industry road-hazard systems separate three layers:

Layer	Purpose	Without it
Observation	Per-detection signal	30 Hz YOLO → thousands of duplicate rows
Hazard entity	Deduplicated geo defect	Map becomes unusable noise
Projection	Map, heatmaps, dashboards	Database rows never become user value

Pipeline: observation → promotion → persistence → projection

3. Architecture overview



4. Storage strategy (no NVMe in BOM)

4.1 Key concepts

Component	What it is	Survives power loss?	Cost
/dev/shm	Software RAM disk (tmpfs)	No	\$0
SD card	Pi boot media	Yes	Already in BOM
NVMe HAT + M.2 SSD	PCIe flash storage	Yes	~\$25–40 (not in BOM)
PostGIS server	Cloud VPS or bench PC	Yes	\$0–20/ mo

Component	What it is	Survives power loss?	Cost
18650 UPS	Battery + POWER_FAIL GPIO	N/A (runtime, not storage)	~\$15–40

Important: `/dev/shm` is **not** crash-safe RAM. It is fast volatile buffer. Original VIGIA design uses **UPS + emergency uplink** (Phase 5 Anti-Death) for power-loss egress, not local durable storage.

4.2 Approved approach (M7.0 — no extra hardware)

1. **Hot buffer:** Pre-allocated event ring in `/dev/shm/vigia-events/` (~250 KB–2 MB)
2. **Continuous sync:** Async thread POSTs batches to server every ~5 s or N events
3. **Canonical store:** Postgres + PostGIS on server
4. **Graceful shutdown:** `systemd` stop hook drains queue before reboot
5. **Power loss (Phase 5):** UPS flush of unsynced queue in ~10–15 s window

Do not log hazard events to the SD card — wear, corruption risk, and explicit architecture ban.

4.3 When to add local SSD (optional M7.1)

Add **USB SSD** (~\$20–35) or **NVMe HAT** (~\$25–40) only if field testing shows:

- Long offline driving without network
- Cold LTE connect too slow for UPS flush
- Need crash-safe local WAL before sync

4.4 UPS vs NVMe

Scenario	Server sync	UPS helps?	NVMe helps?
Normal driving (Wi-Fi/LTE up)	Continuous upload	N/A	Optional
Controlled power cut	Flush in UPS window	Yes	Backup if network slow
Kernel panic	Only already-synced data	No	Yes (if WAL fsync'd)
Hours offline then power cut	Lost unless synced earlier	Last seconds only	Full backlog

5. Edge implementation (Pi C++)

5.1 EventPromoter

Runs after full async fusion (YOLO + MiDaS + temporal + sensors):

1. **RRI gate:** `finalConfidence >= 0.75`
2. **GPS gate:** valid fix (`fix_type >= 2`, `hdop <= 2.5`) or bench override

- 3. **Spatial-temporal dedup:** ~5 m + ~30 s window — reinforce, don't duplicate
- 4. **Async handoff:** SPSC ring → sync thread (never block fusion hot path)

5.2 HazardObservation struct (proposed)

```
struct HazardObservation {
    uuid event_id;           // UUIDv7
    uint64_t frame_index;
    uint64_t timestamp_us;
    std::string device_id;
    float rri, iss, yolo_conf, geometry_conf, temporal_conf;
    cv::Rect bbox;
    double lat, lon;
    float speed_ms, hdop;
    uint32_t gps_fix_type;
};
```

5.3 New modules

File	Role
include/hazard_event.hpp	Struct + JSON serializer
include/event_promoter.hpp	Threshold, dedup, queue
include/event_store.hpp	RAM ring + sync
include/event_signer.hpp	Canonical JSON sign before upload
src/coordinator.cpp	Wire promoter after fusion

6. Server implementation (PostGIS + FastAPI)

6.1 Database tables

- observations** — append-only ingest (idempotent on event_id)
- hazard_entities** — deduplicated map features (spatial merge across fleet)
- device_registry** — fleet identity, last sequence (anti-replay)

6.2 API endpoints

Endpoint	Purpose
POST /v1/events	Signed ingest
GET /v1/hazards?bbox=...	Map viewport
GET /v1/hazards/{id}	Detail + history
GET /v1/hazards/heatmap?bbox=...	Density buckets
GET /v1/devices/{id}/events	Fleet view

6.3 Hazard map UI

Minimal **MapLibre GL JS** app in `web/hazard-map/` — pins by severity, cluster at low zoom, filter by `trust_level`.

7. Security (build in M7 — do not defer)

Crowdsourced hazard maps fail when ingest is open. **Authenticate from the first event.**

7.1 M7 requirements (software trust)

Control	Implementation
Transport	TLS 1.3 only
Device auth	mTLS client certificate (dev: signed requests)
Anti-replay	Monotonic <code>device_seq</code> + timestamp window
Integrity	HMAC/Ed25519 over canonical JSON
Trust labeling	<code>trust_level</code> : unverified → software_signed → hardware_attested
Rate limits	Per device (e.g. 60 events/min)

7.2 Deferred to M3/M4/M6 (hardware trust)

- ATECC608A ECDSA over 96-byte `EtHashInput`
- Pico→Pi AEAD binary packets
- Full server ECDSA verify pipeline
- Multi-device corroboration for public map promotion

7.3 Map trust policy

<code>trust_level</code>	Public map
unverified	Fleet/dev view only
software_signed	Provisioned fleet devices
hardware_attested	Full promotion (M6+)
corroborated	Multiple devices agree

7.4 Example event envelope

```
{
  "event_id": "uuid-v7",
  "device_id": "vigia-dev-001",
  "device_seq": 99102,
  "observed_at": "2026-06-17T14:32:01.042Z",
  "payload_hash": "sha256-hex",
  "signature": "base64-hmac",
```

```
"signed_et": null,  
"trust_level": "unverified"  
}
```

8. Configuration (proposed)

```
device_id: "vigia-dev-001"  
rri_threshold: 0.75  
dedup:  
  radius_m: 5.0  
  window_s: 30.0  
storage:  
  mode: ram_sync  
  event_ring_dir: "/dev/shm/vigia-events"  
  sync_state: "/dev/shm/vigia-sync.state"  
sync:  
  endpoint: "https://api.example.com/v1/events"  
  batch_size: 50  
gps:  
  require_valid: true  
  max_hdop: 2.5
```

9. Delivery phases

M7a — Edge event productization

- HazardObservation struct + JSON schema v1
- EventPromoter + Coordinator wiring
- RAM ring + async sync

M7b — Secure server + ingest

- PostGIS schema + Docker dev stack
- mTLS + anti-replay + signature verify
- device_registry + provisioning script

M7c — Hazard map

- REST geo API + MapLibre UI
- Sync agent with signed payloads
- trust_level map filters

M7d — Hardening (later)

- UPS emergency flush (Phase 5)
 - Optional USB SSD local WAL (M7.1)
 - ATECC hardware attestation (M3/M4/M6)
-

10. Testing

Test	Validates
event_promoter_test	Dedup, RRI 0.75 gate, GPS gate
event_store_test	Ring overflow, sync cursor
ingest_api_test	Idempotency, replay/tamper rejection
security_ingest_test	mTLS + signature + seq window
hazard_merge_test	Cross-device spatial merge
Field run	Pothole appears on map after sync

11. Success criteria

- RRI ≥ 0.75 + valid GPS $\rightarrow$ one deduplicated hazard per 5 m / 30 s window
- Synced events durable on server; hazard map shows geo pins
- Ingest rejects unauthenticated, replayed, tampered requests
- Public map respects trust_level policy
- Fusion hot path: zero new heap allocations

12. Non-goals for M7

- ROS 2 migration (design for it; implement plain C++ first)
- NVIDIA Cosmos 3 integration
- Public anonymous reporting
- Full ATECC on every event (schema ready; hardware is M3/M4)
- Custom vector tile server

13. Research pointers for teammates

Topic	Starting points
PostGIS spatial queries	ST_DWithin, geography types, bbox fetch
Event dedup / clustering	H3 geohash, DBSCAN on geo points
mTLS device provisioning	Smallstep, OpenSSL, AWS IoT cert patterns
Hardware attestation	Microchip ATECC608A, EtHashInput in 03_pico2_firmware_contracts.md
Fleet telematics trust	Waze sybil problems; HERE/map provider corroboration models
Edge buffer patterns	WAL, idempotent ingest, at-least-once delivery

